Name : Vaishnavi Eknath Avhad
Class : D20C
Roll No. : 2

Practical 1

Aim : Cryptography in Blockchain, Merkle root Tree Hash

Theory

Cryptography plays an important role in blockchain by providing security, data integrity, and tamper detection. A cryptographic hash function converts input data into a fixed-length hash value. In this experiment, SHA-256 is used through Python's hashlib library.

A nonce is a value that is changed repeatedly to find a hash satisfying a specific condition, such as having a required number of leading zeros. The successful nonce is called the Golden Nonce in this experiment and demonstrates the basic concept of Proof of Work.

A Merkle Tree is a tree structure used to efficiently represent multiple transactions using cryptographic hashes. Transactions are hashed individually and then combined and hashed repeatedly until a single hash remains, called the Merkle Root. If any transaction is modified, the Merkle Root also changes, helping maintain data integrity.

Thus, hashing, nonce-based cryptographic puzzles, and Merkle Trees are important concepts used to provide security, integrity, and efficient verification of transactions in blockchain.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given stri

XXXXXXX #error statement; 3 Error

[1] ✓ 37s

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                         #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                       #error statement
```

```
Enter a string: Vaishnavi Avhad
Hash: 9880e6c88bd7c71f4941997010ea1d8070525b85ac88041cc5a0c0c828546e84
```

Program #2: Program to generate required target hash with input string and nonce

```
[2]
✓ 11s
```

```python
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                        # Error
```

```
Enter a string: Vaishnavi Avhad
Enter the nonce: 14
Hash Code: 306acca499128656a4bf18b0292e38f39138eebbd16f11d2e7095f536564c976
```

Program 4: Generating Merkle Tree for given set of Transactions

```
[6]
✓ 0s
```

```python
import hashlib

def build_merkle_tree(transactions):                                  # Error
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)                    # Error

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = [
    "Vaishnavi → Bhoomi : $200",
    "Bhoomi → Gargi : $500",
    "Gargi → Vaishnavi : $100",
    "Vaishnavi → Gargi : $300"
]
merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)                                    # Error
```

```
Merkle Root: 041453cbb3d17ef9db70d8414776f93ad365fd01b571d6f68e14c5000959bb5f
```

Conclusion

The experiment helped us understand the basic concepts of cryptographic hashing, SHA-256, nonce, Proof of Work, and Merkle Trees in blockchain. We implemented programs to generate hashes, find a suitable nonce using leading zeros, and construct a Merkle Tree from transactions. The experiment demonstrated how cryptographic techniques ensure data integrity, security, and efficient transaction verification in blockchain.